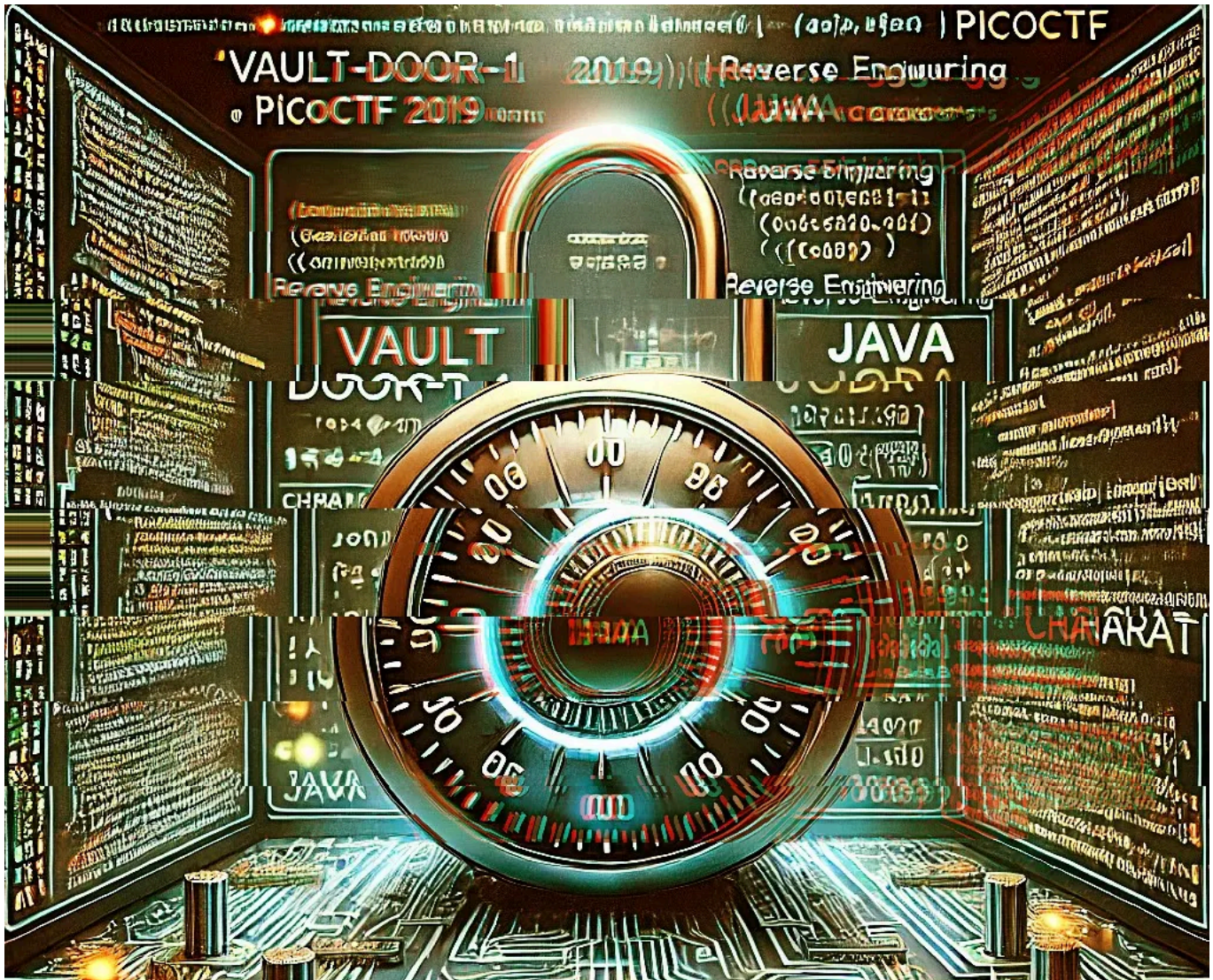


✦ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



# PicoCTF Writeups: vault-door-1



SoBatista

Follow

3 min read · Nov 29, 2024



The “Vault Door 1” challenge builds upon foundational concepts introduced in the **Vault Door Training** challenge, which serves as an excellent starting point for understanding the basics of reverse engineering Java-based puzzles. If you’re unfamiliar with Vault Door Training, check out my write-up [here on Medium](#).

In Vault Door Training, we analyzed simple conditional checks to verify a password. Now, in **Vault Door 1**, the difficulty scales up a bit. This is a fantastic opportunity to further develop our reverse engineering skills while tackling Java code logic in a hands-on way. Let’s go!

## Challenge Overview:

- **Event:** PicoCTF 2019
- **Challenge Name:** vault-door-1
- **Category:** Reverse Engineering
- **Difficulty:** Medium
- **Link:** [here](#)

## Challenge Description

*This vault uses some complicated arrays! I hope you can make sense of it, special agent. The source code for this vault is here: [VaultDoor1.java](#)*

**Hint:** Look up the `charAt()` method online.

Open in app ↗



Medium



Search



Write



I began by downloading the Java file *VaultDoor1.java*. Here's a snippet of the given file:



Interesting! We have the main method and the *checkPassword* method called in main to decide whether a user has or not access depending on the password provided.

The goal is to analyze and reverse-engineer the *checkPassword* method to extract the password. The provided code includes multiple conditions that the input string (our “password”) must satisfy to grant access.



My quick analyzes of the method verifies that:

1. The password is exactly 32 characters long.
2. Each character at specific positions (like the first, second, etc.) must match a predefined value (e.g., the first character, in position 0, must be ‘d’).

The idea behind the challenge is to satisfy every condition simultaneously, Since the conditions are linked with *and* operators, just one incorrect condition will result in a *False* outcome, preventing access.

## Decoding the Password



My first step was to analyze and fully understand each condition in the *checkPassword* method. For instance:

- *password.charAt(0) == 'd'* tells us that the first character of the password is d.
- *password.charAt(29) == 'a'* tells us that the 30th character is a.

Piece by piece, these conditions reveal the complete password. While automation through scripting is an option, I decided to go old school — manually piecing together the solution.

## Final Flag (in base64 encoding):

After I carefully reverse engineered all the conditions, I got the password:

```
cGljb0NURntkMzVjcjRtYmwzX3RIM19jSDRyNGNUM3I1X2Y2ZGFmNH0K
```

**Note:** To decode run:

```
echo -n "encoded_string" | base64 --decode
```

## Real-World Application

This challenge was a fantastic exercise in understanding Java, particularly its string manipulation methods like `charAt()`. It also highlighted the importance of meticulous code analysis and attention to detail — one of the

most crucial skills for reverse engineering tasks. In real-world scenarios, similar approaches are used to analyze and deconstruct obfuscated code or crack authentication mechanisms.

## Final Thoughts

Challenges like **vault-door-1** are excellent to improve both analytical and programming skills. *It is not easy for me yet, but I can see myself improving and growing in reverse-engineering and programming.* If you're tackling similar challenges, consider scripting solutions for efficiency. However, taking the manual route occasionally trains your focus and builds patience.

**What did you think of this challenge?** Drop a comment with your insights or tips! Stay curious, and I'll see you in the next write-up.

**Happy hacking, and I'll see you in the next write-up!**

Cheers,  
*SoBatista*

Reverse Engineering

Ctf

Picoctf

Hacking

Java



Written by SoBatista

106 followers · 4 following

Follow

Ethical Hacker | Entrepreneur | Content Creator YouTube:  
[youtube.com/@SoBatistaCyber](https://youtube.com/@SoBatistaCyber)

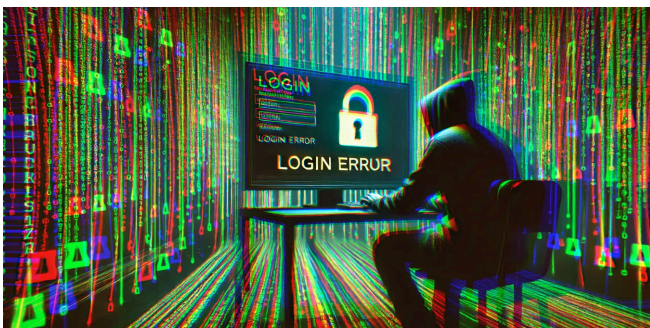
## No responses yet



Suzieq

What are your thoughts?

## More from SoBatista



SoBatista

### PicoCTF Writeups: logon

PicoCTF challenges always find creative ways to test your problem-solving skills, and the...



SoBatista

### PicoCTF Writeup—c0rrupt

Welcome back, CTF enthusiasts! In today's write-up, we tackle the c0rrupt challenge...



Nov 22, 2024 3 2

Jan 29, 2025 12



 SoBatista

### PicoCTF Writeups—dont-use-client-side

Welcome back, CTF enthusiasts! Today, we’re solving the “dont-use-client-side” challenge...

Jan 3, 2025 13



 SoBatista

### PicoCTF Writeup—Glory of the Garden

Challenge Overview:

Oct 25, 2024

See all from SoBatista

### Recommended from Medium

```
#!/bin/sh
# This is a shell archive (produced by GNU sharutils 4.15.2).
# To extract the files from this archive, save it to some FILE, remove
# everything before the '#!/bin/sh' line above, then type 'sh FILE'.
#
lock_dir=sh00046
# Made on 2023-03-16 01:40 UTC by <root@4b9f36a8cccb>.
# Source directory was '/app'.
#
# Existing files will *not* be overwritten, unless '-c' is specified.
#
# This shar contains:
#   length mode       name
#   -----
#   1092 -rw-r--r--   flag
#
MD5SUM=${MD5SUM-md5sum}
f=${MD5SUM} --version | egrep '^md5sum.*(core|text)utils'
test -n "${f}" && md5check=true || md5check=false
${md5check} || \
echo 'Note: not verifying md5sums. Consider installing GNU coreutils.'
```

Cyb3rT4ct1c14n-23

# A Beginner’s Adventure: Cracking the “File types” Forensics Puzzle...

Description: Hey there! Picture this: I stumbled upon a file called Flag.pdf tucked...

Oct 7, 2025



```
1. Check Account Balance
2. Buy Flags
3. Exit

Enter a menu selection
2
Currently for sale
1. Definitely not the flag Flag
2. 1337 Flag
2
1337 flags cost 100000 dollars, and we only have 1 in stock
Enter 1 to buy one1
```

Shao Ray!

# flag\_shop PicoCTF

Hello everyone! This is a writeup on the flag\_shop CTF challenge on PicoCTF.

Nov 4, 2025



ker-2

Forensics picoCTF 2025

VENAX

Hints

problem cannot be solved in the webshell.

1

ption

s learnt about the importance of strong passwords and made sure to the BitLocker drive with a very long and complex password. We d to capture the RAM while this drive was opened however. See if you ask through the encryption!

ColdByte

# PicoCTF: Bitlocker-2

This short write-up outlines two approaches used to solve the PicoCTF challenge...

Nov 22, 2025



Saumya Srivastava

# Flag in Flame — How a Giant Log File Hid a Secret Image and a Flag

by Saumya Srivastava | Forensics | picoCTF 2025

Nov 5, 2025

2



```
Welcome to the Arcane Calculator, GOUGH!

This is the trial version of Arcane Calculator.
The full version may be purchased in person near
the galactic center of the Milky Way galaxy.
Available while supplies last!
=====

__Arcane Calculator__

Menu:
(a) Estimate Astral Projection Mana Burn
(b) [LOCKED] Estimate Astral Slingshot Approach Vector
(c) Estimate Astral Slingshot Approach Vector
```

AUTHOR: NANA AMA ATOMBO-SACKEY

**Description**

Do you know how to use the web inspector?  
Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.  
Its current status is:  
**NOT\_RUNNING**

[Launch Instance](#)

Hints

**1 2**

73,539 users solved

89% Liked

William Lin

**picoCTF: keygenme-py**

This is my personal practice log for picoCTF keygenme-py.

Oct 1, 2025

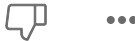


Pawani Wijesekara

**picoCTF—WebDecode Walkthrough**

This guide details the solution for the “WebDecode” challenge, an Easy Web...

Sep 25, 2025



See more recommendations